**I. Estructura y Semántica (HTML)**

**El uso de etiquetas semánticas (como <header>, <nav>, <article>, <footer>) es fundamental. Su propósito principal es dar significado al contenido, lo que mejora enormemente la Accesibilidad para lectores de pantalla y la Indexación para motores de búsqueda (SEO).**

**Para el Diseño Responsive, la etiqueta Meta Viewport (<meta name="viewport" content="width=device-width, initial-scale=1.0">) es esencial. Obliga al ancho de la página a ser igual al ancho del dispositivo y establece el nivel de zoom inicial, permitiendo que las Media Queries de CSS funcionen correctamente.**

**II. Estilo y Layout (CSS)**

**A. Flujo de CSS y Box Model**

1. **Especificidad CSS: Determina qué regla de estilo gana. La regla con mayor especificidad es la que se aplica. El orden de poder es: ID (100) > Clase/Atributo/Pseudo-Clase (10) > Elemento (1). En un conflicto, la regla con mayor puntaje (ej. #id p con 101) anula a las demás.**
2. **Box Model (content-box): Por defecto, el width y height solo definen el área de contenido. El padding y el border se añaden al tamaño final visible. Los márgenes (margin) siempre son espacio externo y no contribuyen al ancho visible del elemento en el diseño.**

**B. Posicionamiento y Unidades**

1. **Unidades Relativas: Se prefieren unidades como rem sobre px (absoluto) por motivos de accesibilidad. rem es más predecible que em porque siempre se basa en el tamaño de fuente del elemento raíz (<html>), evitando problemas de escalado compuesto.**
2. **Mobile First: Los estilos base se escriben para pantallas pequeñas (móviles), y luego se usan Media Queries con min-width para agregar estilos a medida que la pantalla crece.**
3. **Posicionamiento:**
    - **position: absolute;: Se posiciona respecto a su ancestro posicionado más cercano y se mueve con el *scroll* de su contenedor.**
    - **position: fixed;: Se posiciona respecto al viewport (ventana del navegador) y permanece fijo durante el *scroll*.**

4. **z-index: Controla el orden de apilamiento de los elementos, pero solo afecta a los elementos que tienen una propiedad position diferente de static (es decir, posicionado).**

**C. Layout Moderno**

1. **Flexbox**: Diseñado para layouts unidimensionales. Se usa **flex-wrap: wrap;** para permitir que los elementos salten a una nueva línea cuando el espacio se agota.
2. **CSS Grid**: Diseñado para layouts bidimensionales. **grid-template-areas** se utiliza para asignar nombres semánticos a las celdas de la cuadrícula, mejorando la legibilidad del diseño.

**III. Lógica y Flujo de JavaScript**

**A. Variables y Ámbito**

1. **let vs var**: **let** tiene **ámbito de bloque** (restringido a {}), mientras que var tiene ámbito de función o global. La ventaja de let y const es que, si se accede a ellas antes de su declaración explícita (la **Temporal Dead Zone - TDZ**), lanzan un **ReferenceError**, promoviendo un código más seguro.
2. **Coerción de Tipo**: JavaScript realiza **conversión implícita de tipos** al evaluar expresiones (ej. 1 + '2' se convierte en '12'). Por esta razón, se considera una mala práctica usar el operador de igualdad débil ==, ya que puede llevar a resultados inesperados. **Se debe usar el operador de igualdad estricta (===)** para comparar tanto el valor como el tipo de dato.
3. **Orientación a Eventos**: JavaScript es un lenguaje orientado a eventos porque el flujo del programa está determinado por **acciones e interacciones** del usuario o del sistema (clics, carga de DOM, etc.).

**B. Manipulación del DOM y Eventos**

1. **Carga del DOM**: Si el script JS se vincula al final del </body>, el navegador **ya ha construido completamente el árbol DOM**, permitiendo que JavaScript manipule cualquier nodo sin errores.
2. **Creación de Nodos**: document.createElement('div') crea un nodo **en memoria**. Para que sea visible, debe ser **insertado explícitamente** en un nodo padre existente (ej. con appendChild()).
3. **Rendimiento del DOM**: La inserción de muchos nodos uno por uno es ineficiente porque cada operación fuerza al navegador a realizar un **Reflow** (re-cálculo de disposición) y un **Repaint** (re-dibujo), que consumen muchos recursos.

4. **Delegación de Eventos**: Es la técnica **más eficiente** para manejar eventos en múltiples elementos hijos. Consiste en asignar un **solo *listener*** al **contenedor padre** y luego usar **event.target** para identificar el elemento hijo específico que originó el evento.
5. **Detener Flujo de Eventos**: Para prevenir que un evento se propague a los elementos ancestros (*bubbling*), se usa **event.stopPropagation()** o **event.stopImmediatePropagation()**. (Nota: event.preventDefault() solo detiene la acción por defecto del evento, como enviar un formulario).
6. **Shadow DOM**: Característica de los Web Components que permite **aislar el CSS y el marcado** de un componente, previniendo que los estilos globales afecten su interior y viceversa.

**C. Asincronía y Peticiones de Red**

1. **AJAX**: Conjunto de técnicas (usando XMLHttpRequest o Fetch) que permiten la comunicación asíncrona con un servidor **sin recargar la página**.
2. **JSON**: Formato ligero, basado en texto, utilizado como el **estándar para el intercambio de datos** en las peticiones AJAX.
3. **Promesas**: Objeto que gestiona el resultado final (éxito o fracaso) de una operación asíncrona. El estado que indica el éxito es **Fulfilled**.
4. **Manejo de Promesas**:
    - El **Callback Hell** es el anidamiento excesivo de *callbacks* en la asincronía tradicional, mitigado por las Promesas.
    - La sintaxis **async/await** es la forma más moderna e idiomática de trabajar con Promesas, escribiendo el flujo asíncrono de manera síncrona y legible.
    - Se recomienda usar **.catch()** al final de la cadena de Promesas, ya que es el único mecanismo que captura rechazos de la promesa original **y** errores lanzados por cualquier manejador .then() anterior.

**IV. Seguridad y Almacenamiento**

1. **Sanitización de Datos**: Aunque la validación en el cliente (required o JS) mejora la UX, la **sanitización y validación en el lado del SERVIDOR es OBLIGATORIA** para la seguridad (previniendo XSS).
2. **localStorage (Inseguridad)**: **No debe usarse para almacenar datos sensibles** (como tokens de sesión), ya que es vulnerable a ataques de **XSS** (Cross-Site Scripting), permitiendo que el código JS malicioso acceda a todos los datos sin restricción.